

잠긴 플랜두씨 다이어리

내 기록을 나만 보게 만들기 — 가입·로그인·서버 세션·사용자별 자료 차단

- 과제 6 다이어리를 지우지 않고 같은 저장소·배포·DB에서 이어 개발
- bcrypt(cost 12) 비밀번호 · DB에 SHA-256만 남는 폐기 가능한 서버 세션
- 세션에서만 사용자를 정하고, 남의 자료는 양방향 모두 404
- 5일 실제 사용으로 계획 규칙 변경 전후의 시간 오차율 비교

<https://plan-do-see-diary.vercel.app>

2026.09.03

무엇을 보여 드리는가

설계 근거 → 구현 → 검증 → 실제 사용 결과 → 한계 순서로 이어집니다.

설계와 구조

- 03 결과물과 고정 소스
- 04 과제 6에서 이어받은 것
- 05 기술 스택과 계층 구조
- 06 데이터베이스 여덟 개 표
- 07 무중단 4단계 마이그레이션
- 08 인증 설계 결정 네 가지

인증 구현

- 09 설명서 ① ② ③ — 무엇으로·왜·흐름
- 10 가입·로그인·로그아웃 흐름
- 11 비밀번호 규칙과 중복 확인
- 12 세션 수명주기
- 13 소유권 강제 구조
- 14 응답 DTO 계층
- 15 설명서 ⑤ ⑥ — 규칙과 못 막은 것

검증과 결과

- 16 자동 검사 28건
- 17 공개 배포 검증 결과
- 18 30초 점검 목록
- 19 5일 사용 결과
- 20 손계산 대조
- 21~24 실제 화면 증거
- 25 트러블슈팅
- 26 결정 기록
- 27 AI와 남은 제한

제출 결과물과 고정 소스

심사자가 계정 없이 열 수 있는 공개 주소와, 그 시점의 소스를 가리키는 고정 URL.

28건

자동 검사 통과

0건

타 계정 자료 유출

0건

비밀값 노출 (로컬·Git 이력)

404

남의 자료 요청 응답

주소

- 공개 결과물 — <https://plan-do-see-diary.vercel.app>
- GitHub 저장소 — <https://github.com/stulss/plan-do-see-diary>
- 인증 구현 설명서 — docs/과제7/인증_구현_설명서.md
- 검증 안내서 — docs/과제7/검증안내서.md · AI 3줄 — docs/과제7/AI_3줄.md
- 5일 사용 기록 — docs/과제7/5일_사용기록.md · 증거 색인 — docs/과제7/evidence/README.md

제출 시점 고정 URL

github.com/stulss/plan-do-see-diary/commits/db812fcd548f19cc4e4da82ea06063f40e21ee87/ (문서 포함 전체 상태)
github.com/stulss/plan-do-see-diary/commits/67698889bfa7afbb3be1add85e39df65791c29b7/ (전체 자료 API DTO 보강)

과제 6을 지우지 않고 같은 자리에서 이어 만들었다

새 프로젝트를 파는 대신 기존 자료를 보존하고 소유자를 붙이는 쪽을 골랐다.

그대로 이어 쓴 것

- 같은 GitHub 저장소와 main 브랜치
- 같은 Vercel 프로젝트와 배포 주소
- 같은 Supabase PostgreSQL 데이터베이스
- 계획·할 일·실행 기록·돌아보기 구조 전부

지우지 않기로 정한 것

- 과제 6 문서·보고서·증거 (docs/, reports/)
- 과제 6 진행 기록 작업내역_체크리스트.md
- 기존 계획 2건·할 일 5건·돌아보기 1건
- 과제 7 산출물은 docs/과제7/ 에 따로 쌓았다

이력으로 증명한 연속성

과제 6 최종 승인본의 고정 commit 356a46034e94084caec3e17eaaf2ab8d98cef7ce 가 현재 main 이력의 조상인지를 `git merge-base --is-ancestor` 로 확인했다. 과제 7의 모든 커밋은 그 위에 쌓여 있다 (T07-C78).

기존 자료는 사용자가 직접 만든 소유자 계정에 귀속했고, 귀속되지 않은 행이 0건임을 확인한 뒤에 `user_id` 를 NOT NULL 로 바꿨다.

한 프로젝트 안에 화면과 API 를 두고, 서버만 DB 에 연결한다

브라우저에는 DB 접속 문자열이 내려가지 않는다. 비밀값을 NEXT_PUBLIC_* 로 두지 않는다.

Next.js 16.3.3

App Router · 화면과 API 라우트

React 18.3.1

서버 컴포넌트 중심 렌더링

postgres 3.4.7

매개변수 바인딩 SQL 클라이언트

bcryptjs 3.0.3

비밀번호 해시 (cost 12)

Supabase PostgreSQL

관리형 DB · Asia/Seoul 표시

Vercel

공개 배포 · 서버리스 함수

MVC 계층과 책임

app/

화면·API 라우트

lib/service/

인증 절차

lib/repository/

사용자·세션 조회

lib/domain/

규칙·조건 생성

lib/dto/

응답 화이트리스트

Repository 의 모든 함수가 user_id 를 받게 만들면, 소유자 조건을 빠뜨릴 자리 자체가 없어진다.

표 여덟 개 — 여섯 개는 과제 6, 두 개는 과제 7에서 추가

과제 6 표를 지우지 않고 소유자 컬럼만 덧붙였다.

표	역할	소유자
plan	계획. 제목·기간·우선순위·성공 기준·예상 시간	user_id NOT NULL
plan_revision	계획 수정 이력. DB 트리거가 자동 적재한다	plan 경유
task	할 일. 마감일·시작/마감 시각·태그·소프트 삭제	user_id NOT NULL
task_completion	완료 표시. task_id 가 기본키라 중복이 불가능하다	task 경유
run_log	실행 기록. plan·task 를 절대 수정하지 않는다	task 경유
review	돌아보기. 기간별 다음 행동 한 줄	user_id NOT NULL
app_user (과제 7)	계정. 아이디·닉네임·이메일 lower() 유니크 3개	본인
user_session (과제 7)	서버 세션. 토큰의 SHA-256 해시와 만료 시각만 저장	user_id

새 컬럼이 응답에 자동으로 실려 나가지 않도록, 모든 자료 API 는 lib/dto/ 의 화이트리스트를 거친다.

운영 중인 앱을 깨뜨리지 않고 소유자 컬럼을 붙이는 순서

인증판 배포 전까지 과제 6 앱은 `user_id` 없이 INSERT 한다. 그래서 순서가 중요하다.

1 NULL 허용 추가

`user_id` 를 NULL 허용으로 만든다.
기존 앱은 그대로 동작한다.

2 기존 자료 귀속

소유자 계정을 만들고 기존
계획할 일·돌아보기를 귀속한다.

3 임시 DEFAULT

소유자 id 를 기본값으로 걸어
구버전 INSERT 도 깨지지 않게 한다.

4 NOT NULL

미귀속 0건을 확인한 뒤
NOT NULL 을 적용한다.

배포 직후 5단계

인증판을 배포한 뒤 임시 DEFAULT 를 DROP 했다.
이제 소유자 없이 자료를 만드는 경로가 아예 존재하지 않는다.

`plan·task·review` 세 컬럼 모두 NOT NULL 이고 기본값이 없음을 실제 DB
에서 확인했다.

겪은 문제 — DDL 락

마이그레이션을 연속으로 두 번 시작해 두 프로세스가 DDL 락을 서로 기다리며 끝나지 않았다.

해결: 적용 상태 사전 검사와 `connect_timeout 15초 · lock_timeout 10초 · statement_timeout 60초` 를 걸고, 한 프로세스만 한 트랜잭션으로 실행한다.

네 가지 결정이 과제의 요구를 그대로 만족시킨다

"로그아웃하면 이전 값이 안 통한다"를 만들려면 서버가 세션을 폐기할 수 있어야 한다.

직접 구현한 서버 세션

Auth.js 는 Credentials 를 쓰면 세션이 JWT 로 고정되어 폐기할 수 없다.
Supabase Auth 는 로그아웃 뒤에도 토큰이 만료 전까지 유효하다.

→ 로그아웃 = DB 행 삭제

bcrypt (cost 12)

계정마다 소금값을 자동 생성·포함한다. 순수 JS 라 서버리스에서 네이티브 빌드가 필요 없다.

→ 같은 비밀번호도 해시가 다름

불투명 난수 세션 토큰

쿠키에는 의미 없는 난수만 담고 DB 에는 SHA-256 해시만 남긴다. 서명키가 아예 존재하지 않는다.

→ DB 유출로도 도용 불가

응답 DTO 화이트리스트

DB 행을 그대로 내보내지 않는다. 새 컬럼이 자동으로 실려 나가는 사고를 구조가 막는다.

→ user_id·deleted_at 미노출

무엇으로 붙였고, 왜 그것을 골랐고, 어디를 지나는가

docs/과제7/인증_구현_설명서.md 의 ①~③ 을 그대로 옮겼다 (T07-C127~C129).

① 무엇으로 붙였는가

- 인증 방식 — 직접 구현한 서버 DB 세션
- 비밀번호 — bcryptjs 3.0.3, cost 12
- 앱 — Next.js 16.3.3
- 로그인 입력 — 아이디와 비밀번호

브라우저에는 불투명 세션 값을 HttpOnly 쿠키로 보내고, DB 에는 그 값의 SHA-256 해시와 7일 만료 시간만 저장한다.

② 왜 이 방법을 골랐는가

bcryptjs 는 계정마다 임의 소금값을 자동 포함하고 서버리스에서 네이티브 빌드가 필요 없다. DB 세션은 로그아웃·비밀번호 변경 즉시 행을 지워 이전 세션을 폐기할 수 있다.

검토했지만 고르지 않은 것

- Auth.js Credentials — JWT 세션이라 즉시 폐기가 어렵다
- Supabase Auth — 토큰 만료 전 즉시 차단을 따로 설계해야 한다
- express-session — App Router 에 별도 서버·어댑터가 필요하다

③ 흐름과 핵심 소스

가입 /signup → POST /api/auth/signup
→ service/auth → repository/user

로그인 /login → POST /api/auth/login
→ service/auth → lib/session

로그아웃 POST /api/auth/logout
→ lib/session → repository/session

자료 조회 middleware → lib/session
→ domain/query → lib/dto/records

세 흐름이 공통으로 지키는 것

사용자 ID 는 URL·헤더·요청 본문에서 받지 않고 검증된 세션에서만 정한다.

한 건 조회·수정·삭제도 id 와 user_id 를 함께 조건으로 써서, 남의 자료의 존재를 드러내지 않는 404 를 돌려준다.

검증 기록에는 아이디·이메일·비밀번호·세션 쿠키·DB 접속 문자열 원문을 남기지 않았다. 일회성 검증 계정과 자료는 검증 직후 삭제했다.

가입 · 로그인 · 로그아웃 · 자료 조회가 지나는 길

네 흐름 모두 마지막에는 세션과 소유자 조건을 지난다.

가입

- 아이디·닉네임·이메일·비밀번호 4칸
- 규칙 검사를 서버에서 다시 한다
- bcrypt(cost 12) 로 해시해 저장
- DB 유니크 인덱스가 중복을 최종 차단
- 성공하면 곧바로 세션 생성

로그인

- 아이디로 계정을 찾는다
- bcrypt.compare 로 대조
- 아이디가 없든 비밀번호가 틀리든 같은 문구 401
- 난수 토큰 발급, DB 에는 SHA-256 만 저장
- httpOnly-secure-sameSite=lax 쿠키

로그아웃

- 쿠키의 토큰을 해시해 DB 행을 삭제
- 쿠키도 지운다
- 브라우저에 값이 남아 있어도 다시 통하지 않는다
- 비밀번호 변경 시에는 그 계정의 모든 세션을 폐기

자료 조회

- 쿠키 토큰 → SHA-256 → 세션 조회
- 만료 시각을 확인한다
- buildTaskWhere 첫 조건에 user_id 고정
- 응답은 lib/dto/ 화이트리스트를 통과

시도 제한이 없는 대신, 추측 난이도와 중복을 구조로 막았다

규칙 검사는 lib/domain/rules.ts 한 곳에만 두고 서버에서 반드시 다시 검사한다.

비밀번호 규칙

- 10자 이상
- 대문자 · 소문자 · 숫자 · 특수문자 각각 1자 이상
- 어느 조건이 빠졌는지 알려 주고 400 으로 거절
- 화면 검사는 편의이고, 실제 보장은 서버 재검사

중복 확인 — 두 겹

- ① 입력 중 확인 — 아이디·닉네임의 사용 가능 여부만
- ② DB 유니크 인덱스 — lower() 기준 3개
- 확인과 제출 사이에 남이 먼저 가입해도 409 로 막힌다
- 이메일은 확인해 주지 않는다 — 가입자 존재 여부가 새어 나가므로

같은 문구로 답하는 이유

없는 아이디로 로그인해도, 있는 아이디에 틀린 비밀번호를 넣어도 "아이디 또는 비밀번호가 올바르지 않습니다." 한 문구와 401 로만 답한다. 답을 다르게 하면 어떤 아이디가 가입되어 있는지 하나씩 확인할 수 있게 되기 때문이다 (T07-C99).

서버가 언제든지 세션을 없앨 수 있다

쿠키에 든 값은 의미 없는 난수다. DB 에 그 값의 해시가 있어야만 유효하다.

발급

randomBytes(32) 난수
→ 쿠키에 원문
→ DB 에 SHA-256 만

사용

쿠키 토큰을 해시해
DB 조회 · 만료 확인
getSessionUser()

로그아웃

해당 행 1개 삭제
쿠키도 삭제

비밀번호 변경

그 계정의 모든 세션 삭제
다른 기기도 함께 끊긴다

검증한 것

- 로그아웃 전 GET /api/tasks → 200
- 로그아웃 후 같은 쿠키로 재요청 → 401
- 비밀번호 변경 후 이전 세션 → 401
- 이전 비밀번호 로그인 → 401 · 새 비밀번호 → 200

저장·전송 규칙

- 쿠키 httpOnly — 스크립트가 읽지 못한다
- secure (운영) · sameSite=lax · path=/
- 수명 7일, 만료 시각을 DB 에서 함께 확인
- 서명키가 없으므로 키 유출 위험 자체가 없다

사용자는 세션에서만 정해지고, 소유자 조건은 한곳에서 만든다

주소·헤더·요청 본문에서 사용자 ID 를 읽는 코드는 이 프로젝트에 존재하지 않는다.



왜 403 이 아니라 404 인가

- WHERE id = \$1 AND user_id = \$2 가 0행을 돌려주면 자연스럽게 404 가 된다.
- 403 은 "그 자료는 있지만 네 것이 아니다"를 알려 준다 — 존재 자체가 정보가 된다.
- 같은 WHERE 를 UPDATE·DELETE 에도 두었기 때문에, 거절된 요청은 상대 자료를 건드리지 못한다.
- 목록·검색·거르기·집계가 모두 `buildTaskWhere` 한 곳을 지나므로 라우트마다 조건을 빠뜨릴 자리가 없다.

자동 검증 결과 — 두 방향 모두: 읽기 404 · 수정 404 · 삭제 404 · 날짜 이동 404 · 상대 자료 불변 · 목록 유출 0건

DB 행을 그대로 돌려주지 않는다

SELECT * 로 뽑은 행을 그대로 실으면, 나중에 추가한 컬럼이 자동으로 새어 나간다.

고치기 전

```
return NextResponse.json(rows);
```

→ user_id · deleted_at · 내부 시각 컬럼이
응답에 그대로 실렸다.

고친 뒤

```
return NextResponse.json(rows.map(publicTask));
```

→ 화이트리스트에 적힌 필드만 나간다.
새 컬럼은 명시하지 않는 한 나가지 않는다.

적용 범위와 발견 경위

- publicPlan · publicPlanRevision · publicTask · publicCompletion · publicRun · publicReview · publicUser 일곱 가지.
- GET /api/tasks 가 DB 행 전체를 반환하던 누출을 발견해 고치고, 같은 유형이 있던 계획 목록·상세·수정 이력·할 일 생성/상세/수정·실행 기록·돌아보기·내보내기까지 일괄 보강했다.
- 공개 배포 응답(T07-A18)에서 user_id·deleted_at 이 실제로 나오지 않음을 확인했다.

지킨 규칙과, 아직 못 막은 것

못 막은 것을 적지 않으면 이 항목은 통과가 아니다 (T07-C130).

⑤ 세션 · 저장 · 보안 세부 규칙

- 세션 쿠키 — HttpOnly · SameSite=Lax · 배포 환경 Secure · 유효기간 7 일
- 서버 저장 — 세션 원문이 아니라 SHA-256 해시만 user_session 에 저장
- 비밀번호 — bcrypt cost 12, 응답 DTO 와 로그에서 제외
- 중복 방지 — 아이디·닉네임·이메일의 대소문자 무시 유니크 인덱스
- 소유권 — plan·task·review 의 user_id NOT NULL, 목록·검색·집계와 단건 변경 모두 적용
- 세션 폐기 — 로그아웃은 현재 세션 삭제, 비밀번호 변경은 그 사용자의 모든 세션 삭제

⑥ 아직 못 막은 것과 위험

- 로그인 시도 횟수 제한·계정 잠금·CAPTCHA 가 없다. 강한 비밀번호 규칙은 추측 난도를 높이지만 무차별 대입 시도 자체를 막지 못한다.
- 중복확인 API 에 호출 횟수 제한이 없어 아이디·닉네임 존재 여부를 반복 조회할 수 있다. 이메일은 이 API 에서 확인해 주지 않아 범위를 줄였다.
- 아이디 찾기와 등록 정보 일치 방식의 비밀번호 재설정은 이메일 소유 확인·일회용 링크가 없다. 등록 정보를 아는 사람이 복구를 시도할 위험이 있어, 운영 서비스라면 메일 인증으로 교체해야 한다.
- 관리자용 세션 강제 종료와 감사 로그가 없다. 계정 이상 징후를 운영자가 추적하기 어렵다.

자동 검사 28건이 무엇을 지키고 있는가

node --test · 외부 프레임워크 없이 표준 러너만 쓴다. 기존 단언을 약화시키지 않았다.

검사 파일	건수	지키는 것
auth.test.mjs	8	비밀번호 해시·세션 발급과 폐기·소유자 조건·DTO 누출 차단
rules.test.js	6	비밀번호 복잡도·아이디/닉네임 형식·서버 재검사
schedule-route.test.js	4	드래그 날짜 이동의 소유권·서버 시간 계산·계획 없는 할 일
planner.test.mjs	3	주 시작 요일(월요일)·일간/주간/월간 범위 계산
time.test.js	3	시작·마감 시각의 분 계산·자정 경계·시작일 ≤ 마감일
recovery.test.js	2	아이디 찾기·비밀번호 변경의 등록 정보 일치 검사
db.test.js	1	DB 연결의 Asia/Seoul 표시 시간대
plan-color.test.js	1	계획별 색상이 표시 집합 안에서 겹치지 않음

함께 돌리는 것 — tsc --noEmit (타입 오류 0) · next build (프로덕션 빌드 성공) · npm audit --omit=dev (취약점 0건)

공개 배포 주소에서 확인한 결과

일회성 계정으로 검증하고 증거를 남긴 뒤 계정과 자료를 정리했다.

검증 항목	결과
미로그인 자료 API	401
공개 첫 화면 · /tasks	선택 화면 200 · 자료 화면 307 → /login
중복 가입	409
같은 비밀번호의 두 저장값	서로 다른 bcrypt 해시 (cost 12)
틀린 비밀번호 · 없는 아이디	같은 문구 · 401
타 계정 읽기 · 수정 · 삭제	양방향 모두 404, 상대 자료 불변
타 계정 목록 유출	0건 (요청 본문의 사용자 필드는 무시)
로그아웃 · 비밀번호 변경 뒤 이전 세션	401
내 자료 내보내기	JSON 200 · 타 계정 유출 0건 · 내부 필드 미노출
계정 삭제	사용자 · 계획 · 할 일 0건, 이전 세션 401
자동 검사 · 타입 · 빌드 · 감사	28건 통과 · 통과 · 통과 · 취약점 0건
비밀값 스캔	로컬 · Git 전체 이력 0건

심사자가 30초 안에 확인하는 순서

새 시크릿 창에서 세 단계 안에 끝난다. 심사자는 제 계정으로 로그인하지 않는다.

① 어디로
plan-do-see-diary.vercel.app
새 시크릿 창에서 연다

② 무엇을
(1) 주소 열기 → (2) /tasks 붙여 열기
→ (3) 가입 또는 로그인

③ 통과 기준
(1) 선택 화면 (2) 로그인 화면
(3) 뒤에만 그 계정의 다이어리

④ 안 될 때
자료 주소는 로그인 화면으로 돌아감
틀리면 한 문구만 나온다

#	확인	통과 기준	결과
2	로그인 없이 /tasks 열기	자료 대신 로그인 화면	307 → /login
6	중복확인을 지나쳐 그대로 제출	DB 유니크 인덱스가 거절	409
9	없는 아이디로 로그인	8번과 완전히 같은 문구.상태코드	같은 문구 · 401
11	로그아웃 뒤 같은 쿠키로 요청	거절	401
12	다른 계정의 할 일 주소를 직접 열기	찾지 못함	양방향 404
13	목록 응답 전체 확인	남의 자료 0건	양방향 유출 0건
14	내보내기	내 자료만 든 파일 1개	JSON 200 · 유출 0
17	다른 계정의 할 일을 날짜 칸으로 이동	자료 변화 없이 거절	404 · 불변

전체 27개 항목은 docs/과제7/검증안내서.md 에 있다.

계획 규칙을 하나 바꾸고 전후를 같은 지표로 비교

질문: 계획한 시간과 실제로 쓴 시간의 차이를, 계획 규칙을 바꿔서 줄일 수 있는가?

지표 하루 시간 오차율 (%) = (그날 완료한 할 일의 실제 분 합 - 예상 분 합) ÷ 예상 분 합 × 100 · 소수 첫째 자리 반올림

일차	날짜	예상	실제	오차율
1일차	2026-08-29	3 분	4 분	+33.3 %
2일차	2026-08-30	62 분	4 분	-93.5 %
3일차	2026-08-31	4 분	4 분	0.0 %
4일차	2026-09-01	4 분	4 분	0.0 %
5일차	2026-09-02	8 분	4 분	-50.0 %

계획 규칙 변경

2026-08-31 10:30:55 (Asia/Seoul)

2일차 기록 뒤 · 3일차 기록(15:30:23) 앞

예상 시간 상한 60분 → 90분 하나만 변경

-30.1 %

변경 전 평균 (1·2일차)

-16.7 %

변경 후 평균 (3~5일차)

-22.0 %

5일 전체 평균

감추지 않은 것

개선의 대부분은 2일차 한 건(-93.5%)이 빠진 효과다. 2일차는 60분 상한에 맞추려고 62분으로 부풀려 잡았다가 4분에 끝난 날이라, 규칙 변경의 효과라기보다 그날의 어림 실패가 컸다. 하루 1건씩 5일뿐이므로 이 결론은 잠정이다.

화면 합계와 손으로 더한 값이 같은지 대조

돌아보기 화면은 완료일이 아니라 마감일로 거른다. 5일치 마감일이 8/31~9/4 라 기간을 9/4 까지 잡아야 5건이 모두 잡힌다.

플랜두씨

플래너 계획 할 일 돌아보기 내보내기 계정

test님 [로그아웃](#)

SEE

돌아보기

계획과 실제의 차이를 확인하고 다음 행동 한 줄을 정하세요.

100% 완료율

PERIOD

초회 기간

시작일

2026-08-29

→

종료일

2026-09-04

조회

SUMMARY

2026.08.29 - 2026.09.04

숫자를 누르면 근거 목록으로 이동합니다.

계획

5

마감이 기간 안인 할 일

완료

5

완료율 100%

지연

0

미완료·마감 지남

막힘

1

막힌 이유가 있는 기록

예상

81분

계획한 총 시간

실제

20분

기록한 총 시간

차이

-61분

실제 - 예상

NEXT PLAN

다음에 바꿀 한 가지

결과를 평가하는 데서 끝내지 말고 다음 계획에서 실행할 행동으로 바꾸

고칠 점

예: 시작 전에 방해 요소 하나 치우기

이 한 줄로 계획 만들기

T07-A17 · 공개 배포 돌아보기 (기간 2026.08.29~2026.09.04)

손 계산

예상 = 3 + 62 + 4 + 4 + 8 = 81
실제 = 4 + 4 + 4 + 4 + 4 = 20
차이 = 20 - 81 = -61

항목	화면	손
예상 분 합	81	81
실제 분 합	20	20
차이	-61	-61

세 항목 모두 일치

평균 오차율(-22.0%)은 이 화면에 없는 값이다.
화면은 예상·실제·차이 세 개만 보여 주므로
평균은 손 계산으로만 구했고 그 사실을 문서에 적었다.

로그인 전, 계정 없이 열리는 화면

심사자가 계정을 만들지 않고도 여기까지는 볼 수 있다 (T07-C01 / C03).



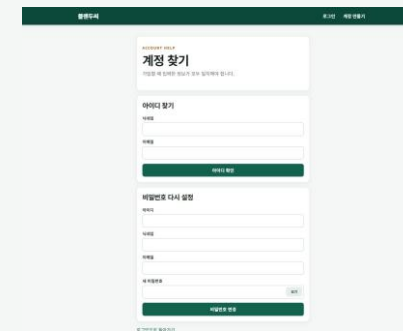
T07-E01 · 공개 시작 화면 (로그인·회원가입 선택)



T07-E02 · 로그인



T07-E03 · 회원가입 (아이디·닉네임·이메일·비밀번호)



T07-E04 · 아이디·비밀번호 찾기

같은 자료를 일간 · 주간 · 월간으로

로그인한 계정의 자료만 나온다. 할 일을 끌어 놓아 날짜를 바꿔도 소유자 조건이 강제된다.

플랜두피

플래너 · 계획 · 할 일 · 들어보기 · 내보내기 · 계정

test님 로그아웃

로그인한 계정의 자료만 보입니다. 공용 PC에서는 사용 후 로그아웃하세요.

PLAN · DO · SEE

9월 2일 수요일

기간을 바꿔 계획과 실제 기록을 한눈에 살펴보세요.

1월 1일 1완료 4분 예상 0분 실제

일간 주간 월간

할 일을 원하는 날짜로 끌어 놓으세요. 키보드: 모바일에서는 할 일 상세에서 날짜를 수정할 수 있습니다.

인행 계획 [\(아이 test test\)](#)

QUICK ADD

플래너에서 바로 추가

2026.09.02 날짜가 기본값으로 들어갑니다.

새 계획 만들기

기간과 설정 가능부터 정합니다.

새 할 일 만들기

계획을 설정 가능한 한 단계로 나눕니다.

02

수요일

2026.09

오늘의 할 일 [\(간\)](#)

test님3

시간 적지않 · 예상 4분

아이 test test · 실제 0분

전체 할 일 보기

T07-E08 · 일간

플랜두피

플래너 · 계획 · 할 일 · 들어보기 · 내보내기 · 계정

test님 로그아웃

로그인한 계정의 자료만 보입니다. 공용 PC에서는 사용 후 로그아웃하세요.

PLAN · DO · SEE

2026.08.31 – 2026.09.06

기간을 바꿔 계획과 실제 기록을 한눈에 살펴보세요.

5월 1일 5완료 20분 예상 0분 실제

일간 주간 월간

할 일을 원하는 날짜로 끌어 놓으세요. 키보드: 모바일에서는 할 일 상세에서 날짜를 수정할 수 있습니다.

인행 계획 [\(아이 test test\)](#)

QUICK ADD

플래너에서 바로 추가

2026.08.02 날짜가 기본값으로 들어갑니다.

새 계획 만들기

기간과 설정 가능부터 정합니다.

새 할 일 만들기

계획을 설정 가능한 한 단계로 나눕니다.

월 31 화 1 수 2 목 3 금 4 토 5 일 6

test님3

시간 적지않 · 예상 3분

test님3

시간 적지않 · 예상 5분

test님3

시간 적지않 · 예상 4분

test님3

시간 적지않 · 예상 4분

test님3

시간 적지않 · 예상 4분

비어 있음

비어 있음

전체 할 일 보기

T07-E09 · 주간 (드래그로 날짜 이동)

플랜두피

플래너 · 계획 · 할 일 · 들어보기 · 내보내기 · 계정

test님 로그아웃

로그인한 계정의 자료만 보입니다. 공용 PC에서는 사용 후 로그아웃하세요.

PLAN · DO · SEE

2026년 9월

기간을 바꿔 계획과 실제 기록을 한눈에 살펴보세요.

4월 1일 4완료 17분 예상 0분 실제

일간 주간 월간

할 일을 원하는 날짜로 끌어 놓으세요. 키보드: 모바일에서는 할 일 상세에서 날짜를 수정할 수 있습니다.

인행 계획 [\(아이 test test\)](#)

QUICK ADD

플래너에서 바로 추가

2026.08.02 날짜가 기본값으로 들어갑니다.

새 계획 만들기

기간과 설정 가능부터 정합니다.

새 할 일 만들기

계획을 설정 가능한 한 단계로 나눕니다.

월 1 화 2 수 3 목 4 금 5 토 6 일

test님3

시간 적지않 · 예상 3분

test님3

시간 적지않 · 예상 5분

test님3

시간 적지않 · 예상 4분

test님3

시간 적지않 · 예상 4분

test님3

시간 적지않 · 예상 4분

비어 있음

비어 있음

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

1

2

3

4

5

6

7

8

9

10

11

12

13

전체 할 일 보기

T07-E10 · 월간

계획 · 할 일 · 돌아보기와 계정 화면

모두 로그인 후에만 열리고, 세션의 사용자 자료만 담긴다.

T07-E11 · 계획 목록

T07-E12 · 계획 상세와 수정 이력

T07-E13 · 할 일 목록

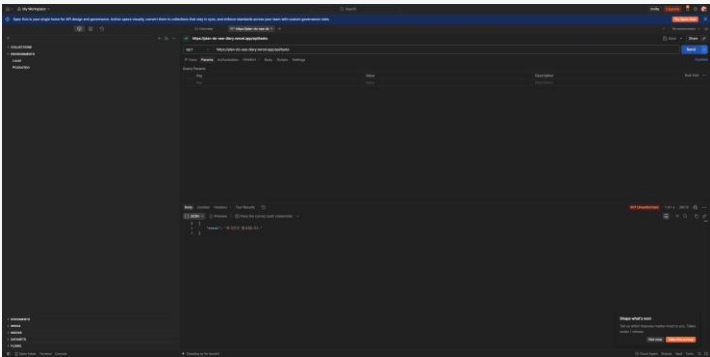
T07-E14 · 할 일 상세와 실행 기록

T07-E15 · 돌아보기

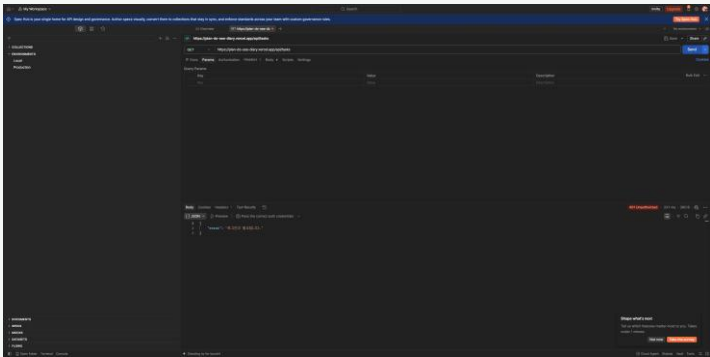
T07-E16 · 계정 (비밀번호 변경·내보내기·삭제)

Postman 요청·응답과 실제 데이터베이스 화면

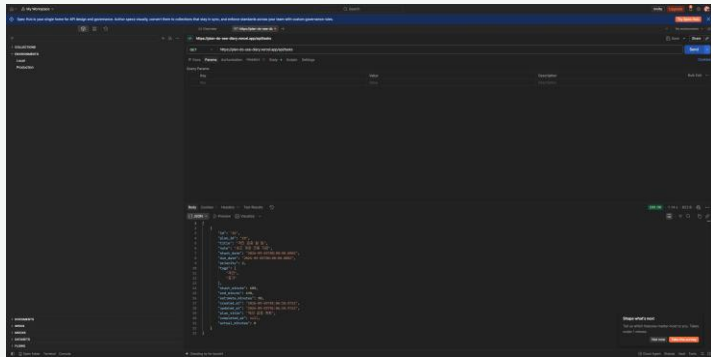
요약 화면이 아니라 실제로 동작하는 화면만 제출 증거로 쓴다. 비밀번호·쿠키·토큰은 모두 가렸다.



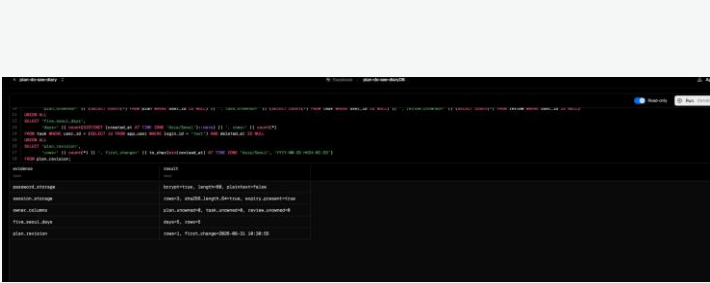
T07-A10 · 미로그인 401



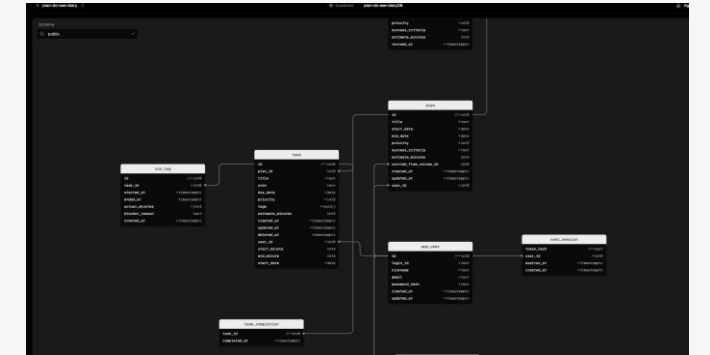
T07-A14 · 로그아웃 뒤 401



T07-A22 · 목록에 본인 자료만



T07-A08 · 실제 DB: bcrypt·세션 해시



T07-A09 · 실제 여덟 개 표 관계



T07-A05 · 규칙 변경 시각과 5일 기록

실제로 막혔던 네 가지와 배운 것

겪지 않은 문제는 적지 않았다. 아래는 모두 이번 과제에서 실제로 발생한 것이다.

DDL 락으로 마이그레이션이 끝나지 않음

두 프로세스가 동시에 스키마를 적용하며 서로의 락을 기다렸다.

해결 · 적용 상태 사전 검사와 연결·락·문장 제한 시간을 걸고 한 프로세스만 실행한다.

DB 시간대를 바꿨는데 UTC 가 계속 보임

ALTER DATABASE 는 성공했지만 풀러가 기존 물리 연결을 재사용했다.

해결 · 설정·검증은 직접 연결로 하고 앱 연결에는 TimeZone 을 명시한다.
timestampz 값에 9시간을 더하지 않는다.

다른 계획인데 색이 같아짐

계획 ID 를 8로 나눈 나머지로 색을 정해, 간격이 8인 계획끼리 겹쳤다.

해결 · 현재 표시 목록 순서로 팔레트를 배정해 최대 여덟 개까지 충돌을 없앴다.

제목을 눌러도 달력이 열림

label 이 날짜 입력을 감싸고 있어 합성 클릭까지 showPicker() 를 실행했다.

해결 · 포인터가 실제 입력 박스에서 시작한 경우에만 달력을 연다.

무엇을 고르고 무엇을 버렸는가

선택이 아니라 과제의 통과 기준과 위험을 근거로 정했다.

결정	근거	버린 대안과 이유
자체 DB 세션	서버가 세션을 폐기할 수 있어야 한다	Auth.js Credentials → JWT 고정, 폐기 불가
bcrypt cost 12	계정마다 소금값 자동 생성·포함	scrypt/PBKDF2 → 소금·형식·비교를 직접 작성
난수 토큰 + SHA-256 저장	DB 가 유출돼도 세션 도용 불가	JWT → 폐기 불가 + 서명키 관리 부담
거절은 404	0행이 자연스럽게 404 가 된다	403 → 자료의 존재를 알려 준다
과제 6 DB 공유	한 저장소·한 배포로 이어지므로 자료가 갈라지면 안 된다	새 DB → 나중에 두 별을 합쳐야 한다
아이디와 이메일 분리	로그인은 아이디로, 표시는 닉네임으로	이메일을 아이디로 → 표시 이름이 이메일로 노출
중복확인 두 겹	화면은 편의, 보장은 DB 유니크 인덱스	화면 확인만 → 동시 가입 시 중복 통과
MVC 계층 + DTO	Repository 가 user_id 를 받으면 빠뜨릴 자리가 없다	계층 없이 유지 → 새 라우트에서 누락 가능

AI가 만든 것, 내가 정한 것, 그리고 남은 구멍

무엇을 못 막았는지까지 적어야 이 과제가 끝난다.

AI와 내 판단 3줄

- AI에게 맡긴 일 — 과제 6 코드 분석, 가입·로그인·서버 세션·사용자별 자료 차단 구현, DB 이전과 자동 검증, 문서 최신화.
- 내가 직접 판단한 일 — 과제 6을 지우지 않고 같은 저장소·배포·DB에서 이어가기로 정했고, 실제 계정 가입과 계획·할 일·실행 기록 입력은 내가 했다.
- AI 제안을 따르지 않은 일 — 새 DB로 분리하는 대안 대신, 과제 6 자료를 보존해 내 계정으로 귀속하는 방식을 택했다.

못 막은 것 (남은 제한)

- 로그인 시도 횟수 제한·계정 잠금·CAPTCHA 가 없다 — 무차별 대입에 취약하다.
- 비밀번호 찾기가 등록 정보 일치 방식이다. 이메일 소유 확인과 일회용 링크가 없다.
- 관리자용 세션 강제 종료와 감사 로그가 없다.

핵심 제출 범위인 가입 · 로그인 · 즉시 세션 폐기 · 사용자별 자료 차단은 구현하고 공개 배포 환경에서 검증했다.

비밀번호 규칙은 10자 이상 + 대문자·소문자·숫자·특수문자 각 1자 이상으로 두어, 시도 제한이 없는 상태에서 추측 난이도를 올렸다. 중복 확인은 화면 검사와 DB 유니크 인덱스 두 겹으로 두어 확인과 제출 사이에 남이 먼저 가입하는 경우까지 막았다.